2. 提示词模版

- 提示词模板的核心作用是将用户输入和参数转换为语言模型可理解的指令。它让你的提示词从硬编码的字符串变成了可复用、可维护的工具。

- 提示词模版主要争对BaseChatModel中的invoke，stream的inputs参数对应设计：
  - PromptTemplate：基于字符串设计的提示词模版。
  - ChatPromptTemplate：基于dict与BaseMessage设计的提示词模版。同时争对不同角色设计更进一步的提示词：
    - AIMessagePromptTemplate
    - HumanMessagePromptTemplate
    - SystemMessagePromptTemplate
    - ChatMessagePromptTemplate
  - MessagesPlaceholder：基于对话历史的列表设计的提示词模版。

- 提示词模版类似于格式字符串参数化，这主要有如下好处：
  - 可复用
    - 一个模板，多次使用
  - 可维护
    - 模板和数据分离
  - 类型安全
    - 自动验证变量
  - 可测试
    - 更容易编写测试
  - 可组合
    - 可以组合多个模板

2.1. PromptTemplate-简单提示词模板

- PromptTemplate使用简单，就是单元的格式字符串参数化。其继承结构如下：
  - PromptTemplate：提供format函数格式化模型。一般不使用构造器构建模版对象，而是使用方便的工程函数，
    - StringPromptTemplate：抽象类。规范了format接口函数
      - BasePromptTemplate：提供了一些通用功能，包含字典提示词，文件保存

- PromptTemplate：提供format函数格式化模型。一般不使用构造器构建模版对象，而是使用方便的工程函数。
  - 对象构造：
    - 类函数：

        - `from_template(template: str, *, template_format: PromptTemplateFormat = "f-string", partial_variables: dict[str, Any] | None = None, **kwargs: Any ) -> PromptTemplate`
        - `from_file(template_file: str | Path, encoding: str | None = None, **kwargs: Any) -> PromptTemplate`
        - `from_examples(examples: list[str], suffix: str, input_variables: list[str], example_separator: str = "\n\n", prefix: str = "", **kwargs: Any) -> PromptTemplate`
    - 格式化生成提示词：
        - `format(self, **kwargs: Any) -> str`
    - 模版运算符：
        - `__add__`

- BasePromptTemplate：主要提供了文件保存，字典与PromptValue等格式转换
    - partial：部分格式化（返回模版，而不是结果）。
    - invoke：格式化为PromptValue类型。
    - dict：格式化为字典。
    - save：保存为文件。

- 简单的使用方式

In [2]:
```
from langchain_core.prompts import PromptTemplate

# 定义模板，{topic} 是待填充的变量
template = "计算{add}+{added}的结果。"
prompt = PromptTemplate.from_template(template)

# 填充变量，生成最终的提示词
final_prompt = prompt.format(add=20, added="30")
print(final_prompt)
```

```
计算20+30的结果。
```

- 构造器构造模版对象

In [8]:
```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate(
    input_variables=["变量1", "变量2"],       # 显式指定变量
    template="提示词模板文本 {变量1} 和 {变量2}"
)
print(template)
print(template.format(变量1="Hello", 变量2="World"))
```

```
input_variables=['变量1', '变量2'] input_types={} partial_variables={} template
='提示词模板文本 {变量1} 和 {变量2}'
提示词模板文本 Hello 和 World
```

In [9]:
```
print(template.input_variables)
print(template.partial_variables)
print(template.input_types)
print(template.template)
```

```
['变量1', '变量2']
{}
{}
提示词模板文本 {变量1} 和 {变量2}
```

- 分多步格式化

In [11]:
```
from langchain_core.prompts import PromptTemplate
template = PromptTemplate.from_template(
    "请计算{var1}{op}{var2}"
)

# 预填充 role
partial_template = template.partial(var1="34")

# 现在只需要提供 task
prompt = partial_template.format(op="加", var2="45")
print(prompt)
```

```
请计算34加45
```

- invoke的使用
  - 返回PromptValue，该对象有

In [17]:
```
from langchain_core.prompts import PromptTemplate
template = PromptTemplate.from_template(
    "请计算{var1}{op}{var2}"
)
prompt_value = template.invoke(
    {
        "var1": 45,
        "op": "加",
        "var2": "99"
    }
)
print(prompt_value)
print(prompt_value.text)
print(prompt_value.to_messages())
print(prompt_value.to_string())
```

```
text='请计算45加99'
请计算45加99
[HumanMessage(content='请计算45加99', additional_kwargs={}, response_metadata=
{})]
请计算45加99
```

2.2. ChatPromptTemplate-聊天提示词模板

- 现代大模型（如 GPT-4、Claude）以对话为核心，区分了 system（系统）、human（用户）和 ai（助手）等角色。ChatPromptTemplate 就是为这种场景设计

的，它将提示词构建成一个结构化的消息列表。

- ChatPromptTemplate的继承结构如下：
  - ChatPromptTemplate：负责模版对象的构建。
    - BaseChatPromptTemplate：负责format格式化。
      - BasePromptTemplate：这个与PromptTemplate一样的父类，负责invoke，save等通用功能。

- ChatPromptTemplate类对象的构建：
  - 构造器
    - `__init__(messages: Sequence[MessageLikeRepresentation],*, template_format: PromptTemplateFormat = "f-string", **kwargs: Any)`
  - from_messages类函数
    - `from_messages(messages: Sequence[MessageLikeRepresentation], template_format: PromptTemplateFormat = "f-string", ) -> ChatPromptTemplate`
  - from_template类函数：(构造一个HumanMessagePromptTemplate)
    - `from_template(cls, template: str, **kwargs: Any) -> ChatPromptTemplate`

In [24]:
```
from langchain_core.prompts import ChatPromptTemplate

# 构建一个包含系统角色和用户角色的对话模板
template = ChatPromptTemplate.from_messages([
    ("system", "你是一位{lang}软件工程师，请用简单易懂的语言解释问题。"),
    ("human", "写一个{alg}程序")
])

# 填充变量，生成用于调用模型的消息列表
messages = template.invoke({"lang": "C++", "alg": "冒泡算法"})
print(messages)
print("*" * 60)
messages = template.format(lang="C++", alg="冒泡算法")
print(type(messages))   # 字符串
print(messages)
template.format_prompt(lang="C++", alg="冒泡算法").messages
```

```
messages=[SystemMessage(content='你是一位C++软件工程师，请用简单易懂的语言解释问题。', additional_kwargs={}, response_metadata={}), HumanMessage(content='写一个冒泡算法程序', additional_kwargs={}, response_metadata={})]
************************************************************
<class 'str'>
System: 你是一位C++软件工程师，请用简单易懂的语言解释问题。
Human: 写一个冒泡算法程序
```

Out[24]:
```
[SystemMessage(content='你是一位C++软件工程师，请用简单易懂的语言解释问题。', additional_kwargs={}, response_metadata={}),
 HumanMessage(content='写一个冒泡算法程序', additional_kwargs={}, response_metadata={})]
```

- 代码说明：

    - format返回的是字符串。

- 上面使用元组是推荐的方式，与可以使用下面方式

In [25]:
```
from langchain_core.prompts import (
    SystemMessagePromptTemplate,
    HumanMessagePromptTemplate
)

system_template = SystemMessagePromptTemplate.from_template(
    "你是一个{role}"
)
human_template = HumanMessagePromptTemplate.from_template(
    "{question}"
)

template = ChatPromptTemplate.from_messages([
    system_template,
    human_template
])
template.invoke({"role": "C++程序员", "question": "写一个冒泡算法"})
```

Out[25]:
```
ChatPromptValue(messages=[SystemMessage(content='你是一个C++程序员', additional
_kwargs={}, response_metadata={}), HumanMessage(content='写一个冒泡算法', addit
ional_kwargs={}, response_metadata={})])
```

In [27]:
```
template.format_prompt(role="C++程序员", question="写一个冒泡算法")
```

Out[27]:
```
ChatPromptValue(messages=[SystemMessage(content='你是一个C++程序员', additional
_kwargs={}, response_metadata={}), HumanMessage(content='写一个冒泡算法', addit
ional_kwargs={}, response_metadata={})])
```

In [28]:
```
template.format_messages(role="C++程序员", question="写一个冒泡算法")
```

Out[28]:
```
[SystemMessage(content='你是一个C++程序员', additional_kwargs={}, response_meta
data={}),
 HumanMessage(content='写一个冒泡算法', additional_kwargs={}, response_metadata
={})]
```

- 代码说明：
    - 上面各种方法的使用，无疑首选format_messages方式格式化。

2.3. MessagesPlaceholder-历史提示词模版

- 这是实现有状态对话的关键。在构建聊天机器人时，模型需要知道之前的对话内容才能进行连贯的交流。MessagesPlaceholder 的作用就是在提示词中预留一个位置，用来动态地、完整地插入过去的对话记录。

- MessagesPlaceholder的继承结构如下：
    - MessagesPlaceholder：主要提供format_messages方法进行格式化。
        - BaseMessagePromptTemplate：提供通用的处理

In [32]:
```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage

# 构建一个包含对话历史占位符的模板
template = ChatPromptTemplate.from_messages([
    ("system", "你是一位友好的客服助手。"),
    MessagesPlaceholder(variable_name="chat_history"), # 对话历史将插入在这里
    ("human", "{input}")
])

# 假设这是之前的对话
chat_history = [
    HumanMessage(content="我的订单什么时候到？"),
    AIMessage(content="您好，您的订单预计明天送达。")
]

# 填充变量，完整的消息列表会包含系统提示、历史对话和当前问题
final_messages = template.invoke({
    "chat_history": chat_history,
    "input": "那我可以修改收货地址吗？"
})
print(final_messages)
final_messages = template.format_messages(
    chat_history=chat_history,
    input="那我可以修改收货地址吗？"
)
print("*" * 50)
print(final_messages)
```

```
messages=[SystemMessage(content='你是一位友好的客服助手。', additional_kwargs={},
response_metadata={}), HumanMessage(content='我的订单什么时候到？', additional_kw
args={}, response_metadata={}), AIMessage(content='您好，您的订单预计明天送达。',
additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=
[]), HumanMessage(content='那我可以修改收货地址吗？', additional_kwargs={}, respo
nse_metadata={})]
**************************************************
[SystemMessage(content='你是一位友好的客服助手。', additional_kwargs={}, response
_metadata={}), HumanMessage(content='我的订单什么时候到？', additional_kwargs={},
response_metadata={}), AIMessage(content='您好，您的订单预计明天送达。', addition
al_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[]), HumanM
essage(content='那我可以修改收货地址吗？', additional_kwargs={}, response_metadat
a={})]
```

2.4. LCEL链式调用

- LCEL = LangChain Expression Language，LangChain 的表达式语言。
  - 链是将多个组件连接在一起的方式，形成处理流程。比如：输入 → 模板 → 模型 → 输出

In [33]:
```
from langchain_core.prompts import ChatPromptTemplate
from langchain.chat_models import init_chat_model

# 创建组件
template = ChatPromptTemplate.from_messages([
    ("system", "你是{role}"),
```

```
    ("user", "{input}")
])

model = init_chat_model(
    model="qwen3.5:9b",          # 你在 Ollama 中下载的模型名称
    model_provider="ollama",      # 关键参数，指定使用 ollama 提供商
    temperature=0.7,              # 其他参数与使用任何其他模型一样
    base_url="http://localhost:11434"  # 可选，如果你的 Ollama 不是默认地址，则需
)


# 使用 | 创建链
chain = template | model

# 直接调用链
response = chain.invoke({
    "role": "数学计算大师",
    "input": "正弦函数在0到π区间的积分等于多少？"
})

print(response.content)
```

这是一个经典的定积分计算问题。作为数学计算大师，我来为你详细推导一下：

我们需要计算的是正弦函数 $\sin(x)$ 在区间 $[0, \pi]$ 上的定积分：

$$ I = \int_{0}^{\pi} \sin(x) \, dx $$

计算步骤如下：

1. **寻找原函数**：
   正弦函数 $\sin(x)$ 的原函数是 $-\cos(x)$。

2. **应用牛顿-莱布尼茨公式**：
   $$ I = \left[-\cos(x) \right]_{0}^{\pi} $$

3. **代入上下限**：
   $$ I = (-\cos(\pi)) - (-\cos(0)) $$

4. **计算三角函数值**：
   * $\cos(\pi) = -1$
   * $\cos(0) = 1$

5. **得出结果**：
   $$ I = (-(-1)) - (-1) $$
   $$ I = 1 + 1 $$
   $$ I = 2 $$

结论：
正弦函数在 0 到 π 区间的积分等于 **2**。

- 如果模型不能干的事情怎么办？比如查询明天天气预报。